c:\hpc-23-feb-2026\.\utils\testcases\test_fill_random.cpp

```
#include<utils.h>
#include<vector>
#include<iostream>
using namespace std;
int main()
{
vector<uint32_t> v;
fill_random(v,10,20,50);
for(auto x:v) cout<<x<<endl;
return 0;
}
```

Thinking Machines Classroom Codes

c:\hpc-23-feb-2026\.\utils\src\utils.cpp

```
#include<vector>
#include<cstdlib>
#include<cstdint>
#include<ctime>
using namespace std;
void fill_random(vector<uint32_t> &v,size_t fill_count,uint32_t min=0,uint32_t max=RAND_MAX)
{
srand(time(NULL)); // setting up seed value
if(v.size()!=fill_count) v.resize(fill_count);
for(int i=0;i<fill_count;++i)
{
v[i]=rand()%(max-min)+min;
}
}
```

Thinking Machines Classroom Codes

c:\hpc-23-feb-2026\.\utils\include\utils.h

```
#pragma once
#include<vector>
#include<cstdint> // instead of stdint.h
#include<cstdlib>
using namespace std;
void fill_random(vector<uint32_t> &v,size_t fill_count,uint32_t min=0,uint32_t max=RAND_MAX);
```

c:\hpc-23-feb-2026\.\cmp.sh

```
g++  -I  ../utils/include  $1.cpp  -L ../utils/lib  -lhpc-utils   -o  $1.out
```

c:\hpc-23-feb-2026\.\23\eg179.cpp

```
#include<iostream>
#include<vector>
#include<utils.h>
#include<chrono>
using namespace std;
int main()
{
vector<uint32_t>  v;
fill_random(v,100000000,10,100);
uint64_t sum=0;
auto start=chrono::high_resolution_clock::now();
for(auto x:v) sum+=x;
auto end=chrono::high_resolution_clock::now();
auto duration=chrono::duration_cast<chrono::milliseconds>(end-start);
cout<<"Sum : "<<sum<<endl;
cout<<duration.count()<<" milliseconds"<<endl;
return 0;
}
```

c:\hpc-23-feb-2026\.\23\eg180.cpp

```
#include<iostream>
#include<vector>
#include<utils.h>
#include<chrono>
#include<numeric>
#include<execution>
using namespace std;
int main()
{
vector<uint32_t>  v;
fill_random(v,100000000,10,100);
uint64_t sum=0;
auto start=chrono::high_resolution_clock::now();
sum=reduce(execution::par,v.begin(),v.end());
auto end=chrono::high_resolution_clock::now();
auto duration=chrono::duration_cast<chrono::milliseconds>(end-start);
cout<<"Sum : "<<sum<<endl;
cout<<duration.count()<<" milliseconds"<<endl;
return 0;
}
```